

LAPORAN TUGAS

**WEB SERVER
(TIF 330805)**

MINGGU KE 2



ACARA 4

DIBUAT OLEH:

NAMA: CINTYA AZWA SAFRINA R.H

NIM:E41250631

GOL:A

**PROGAM STUDY DILUAR KAMPUS UTAMA(PSDKU) POLITEHNIK NEGRI JEMBR
TEHNIK INFORMATIKA KAB.NGANJUK JL. JENDRAL GATOT SUBROTO NO
106,KAUMAN,KECAMATAN NGANJUK,KABUPATEN NGANJUK JURUSAN
TEKNOLOGI INFORMASI POLITEHNIK NEGRI JEMBER TAHUN 2026**

PENDAHULUAN

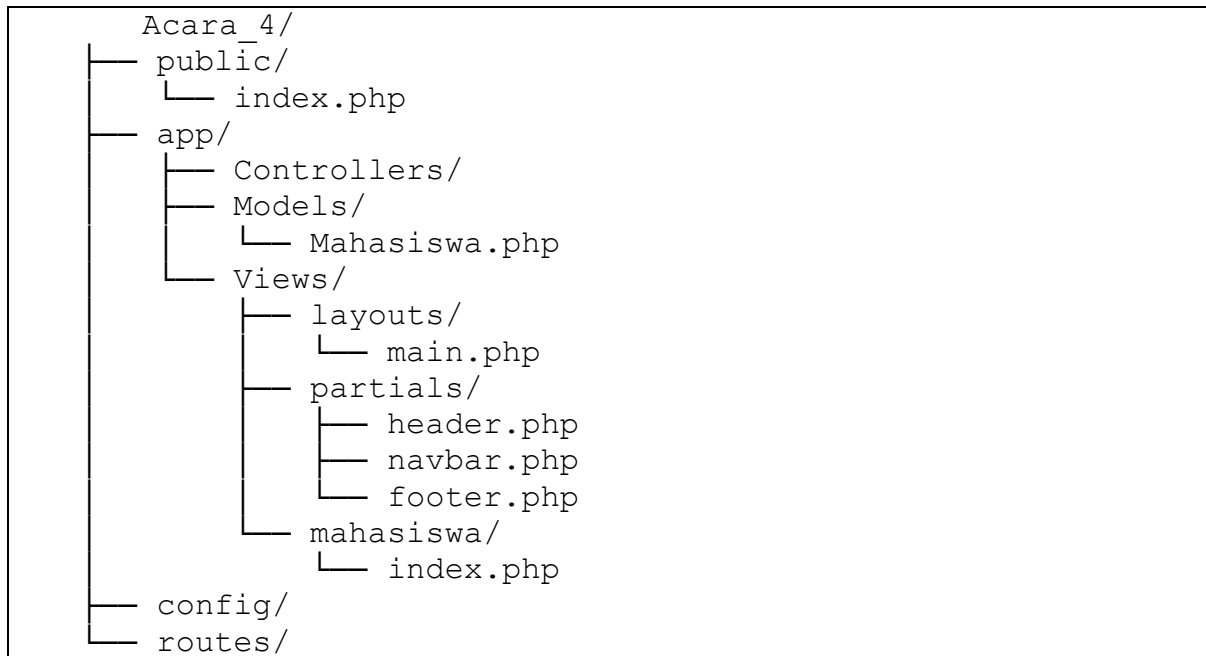
Dalam pengembangan aplikasi web berbasis PHP, seringkali setiap halaman memiliki bagian-bagian yang sama, seperti header (logo dan judul), navbar (menu navigasi), dan footer (informasi hak cipta). Jika bagian-bagian ini ditulis ulang secara manual di setiap file halaman, maka ketika terjadi perubahan (misalnya penambahan menu baru), pengembang harus mengedit banyak file sekaligus, yang tentu tidak efisien dan rawan kesalahan.

Untuk mengatasi hal tersebut, PHP menyediakan fungsi `include` dan `require` yang memungkinkan penyisipan isi satu file ke dalam file lain. Perbedaan mendasar antara keduanya terletak pada penanganan error: `include` hanya akan menampilkan peringatan (*warning*) dan tetap melanjutkan eksekusi script jika file yang dipanggil tidak ditemukan, sedangkan `require` akan menghentikan eksekusi script secara total (*fatal error*) jika file tidak ditemukan. Dengan memanfaatkan konsep ini, halaman web dapat dipecah menjadi bagian-bagian kecil yang dapat digunakan kembali (*partial*), seperti `header.php`, `navbar.php`, dan `footer.php`, yang kemudian digabungkan melalui satu file layout utama.

Selain konsep layout dan partial, praktikum ini juga memperkenalkan dasar-dasar pemrograman berorientasi objek (Object-Oriented Programming/OOP) dalam PHP, khususnya konsep *class* dan *object*. Class berperan sebagai cetak biru (*blueprint*) yang mendefinisikan properti dan perilaku suatu entitas, sedangkan object merupakan hasil instansiasi (penerapan nyata) dari class tersebut. Konsep encapsulation turut diterapkan melalui penggunaan visibilitas `private` pada properti class, yang hanya dapat diakses melalui method tertentu (*getter* dan *setter*), sehingga data tidak dapat diubah secara sembarangan dari luar class. Praktikum ini juga memperkenalkan konsep namespace, yang digunakan untuk mencegah terjadinya konflik penamaan class ketika aplikasi menggunakan banyak class dengan nama yang serupa dari sumber berbeda.

IMPLEMENTASI

1. Membuat struktur folder project baru bernama Acara_4 di dalam htdocs, dengan struktur MVC yang dilengkapi folder layouts dan partials:



2. Membuat file partial header.php, navbar.php, dan footer.php yang berisi potongan HTML untuk bagian yang berulang di setiap halaman.
3. Membuat file layout utama main.php yang menggabungkan seluruh partial menggunakan include, dengan bagian konten dimuat secara dinamis menggunakan require \$content.
4. Membuat class Mahasiswa pada app/Models/Mahasiswa.php dengan namespace App\Models, properti bersifat private (\$nim, \$nama, \$prodi), constructor, beberapa method *getter*, method *setter* dengan validasi, serta method tambahan getAngkatan() sebagai penyelesaian Tugas Mandiri.
5. Membuat beberapa object dari class Mahasiswa di dalam public/index.php, kemudian mengirimkan data tersebut ke View melalui variabel \$daftarMahasiswa dan memuat layout utama.
6. Menampilkan data object dalam bentuk tabel Bootstrap pada app/Views/mahasiswa/index.php, termasuk kolom hasil perhitungan angkatan.
7. Mengakses halaman melalui browser untuk memastikan seluruh partial, layout, dan data object tampil dengan benar.

Struktur Kode

Class Mahasiswa dengan OOP dan Tugas Mandiri (app/Models/Mahasiswa.php)

php

```
namespace App\Models;

class Mahasiswa
{
    private string $nim;
    private string $nama;
    private string $prodi;

    public function __construct(string $nim, string $nama, string $prodi)
    {
        $this->nim = $nim;
        $this->nama = $nama;
        $this->prodi = $prodi;
    }

    public function getNim(): string { return $this->nim; }
    public function getNama(): string { return $this->nama; }
    public function getProdi(): string { return $this->prodi; }

    // Tugas Mandiri: menentukan angkatan dari 2 digit awal NIM
    public function getAngkatan(): string
    {
        $duaDigitAwal = substr($this->nim, 0, 2);
        return '20' . $duaDigitAwal;
    }
}
```

Layout utama menggabungkan partial (app/Views/layouts/main.php)

```
<body>
<?php include __DIR__ . '/../partials/header.php'; ?>
<?php include __DIR__ . '/../partials/navbar.php'; ?>
<main class="container mt-4">
    <?php require $content; ?>
</main>
<?php include __DIR__ . '/../partials/footer.php'; ?>
</body>
```

Entry point membuat object dan mengirim ke View (public/index.php)

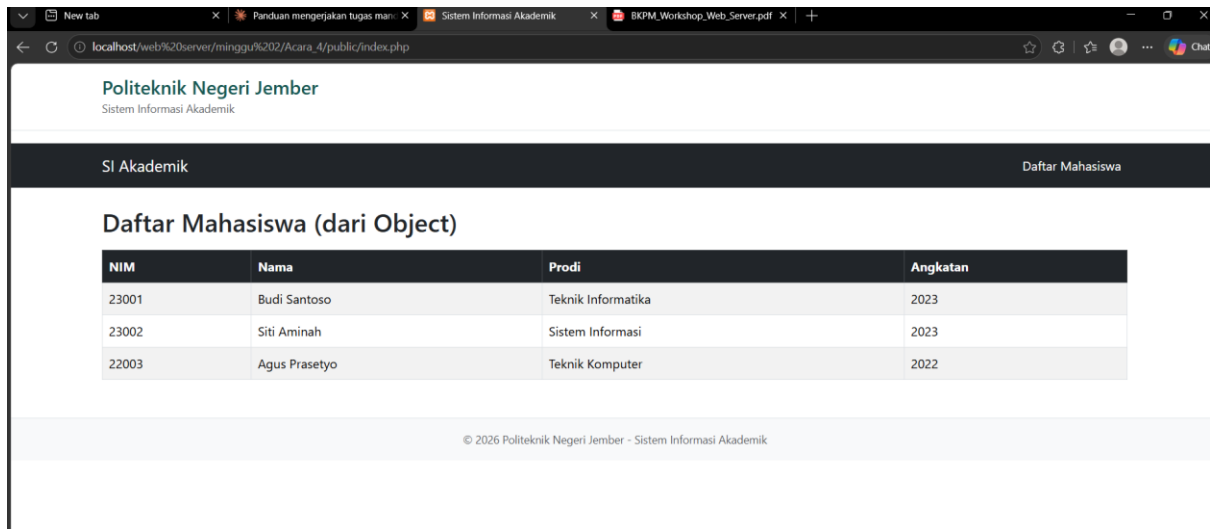
```
require_once __DIR__ . '/../app/Models/Mahasiswa.php';
use App\Models\Mahasiswa;

$daftarMahasiswa = [
    new Mahasiswa("23001", "Budi Santoso", "Teknik
Informatika"),
    new Mahasiswa("23002", "Siti Aminah", "Sistem
Informasi"),
```

```
        new Mahasiswa("22003", "Agus Prasetyo", "Teknik
Komputer"),
];

$content = __DIR__ . '/../app/Views/mahasiswa/index.php';
require __DIR__ . '/../app/Views/layouts/main.php';</form>
```

Hasil



NIM	Nama	Prodi	Angkatan
23001	Budi Santoso	Teknik Informatika	2023
23002	Siti Aminah	Sistem Informasi	2023
22003	Agus Prasetyo	Teknik Komputer	2022

© 2026 Politeknik Negeri Jember - Sistem Informasi Akademik

Analisis

Penggunaan include pada file main.php memungkinkan bagian header, navbar, dan footer ditulis satu kali saja di file terpisah, kemudian dipanggil ulang di layout utama. Dengan pendekatan ini, apabila terjadi perubahan pada navbar (misalnya penambahan menu), pengembang hanya perlu mengedit satu file (navbar.php), dan perubahan tersebut otomatis berlaku pada seluruh halaman yang menggunakan layout tersebut.

Variabel \$content pada main.php berperan penting sebagai penentu file View mana yang akan ditampilkan sebagai konten utama, dengan nilai variabel tersebut diberikan dari file public/index.php sebelum layout dimuat. Teknik ini memungkinkan satu layout digunakan secara fleksibel untuk berbagai halaman konten yang berbeda.

Pada sisi OOP, class Mahasiswa menerapkan **encapsulation** dengan menjadikan properti \$nim, \$nama, dan \$prodi bersifat private, sehingga tidak dapat diakses atau diubah secara langsung dari luar class. Akses terhadap data hanya dapat dilakukan melalui method *getter* (getNim(), getNama(), getProdi()), sementara perubahan data nama harus melalui method setNama() yang menyertakan validasi panjang karakter, sehingga mencegah nama yang tidak valid tersimpan pada object.

Method tambahan getAngkatan() yang dibuat untuk Tugas Mandiri memanfaatkan fungsi substr() untuk mengambil dua digit pertama dari NIM, kemudian menggabungkannya dengan awalan "20" untuk membentuk tahun angkatan empat digit. Hasil pengujian menunjukkan bahwa mahasiswa dengan NIM diawali "23" (23001, 23002) menghasilkan angkatan 2023, sedangkan NIM diawali "22" (22003) menghasilkan angkatan 2022, yang sesuai dengan asumsi format NIM yang diberikan pada modul.

Secara keseluruhan, penggabungan konsep layout, partial, dan OOP pada praktikum ini menunjukkan bagaimana kode program dapat disusun secara lebih terstruktur, mudah dipelihara (*maintainable*), dan siap dikembangkan lebih lanjut untuk terhubung dengan Controller dan database pada pertemuan berikutnya

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Fungsi *include* dan *require* memungkinkan penyisipan isi satu file PHP ke dalam file lain, dengan perbedaan utama pada penanganan error ketika file tidak ditemukan — *include* menampilkan peringatan dan tetap melanjutkan eksekusi, sedangkan *require* menghentikan eksekusi secara total.
2. Pendekatan layout dan partial (header, navbar, footer) membuat kode lebih efisien dan mudah dipelihara, karena perubahan pada satu bagian yang berulang cukup dilakukan di satu file, tanpa perlu mengedit setiap halaman satu per satu.
3. Konsep dasar OOP dalam PHP — class sebagai *blueprint* dan object sebagai *instance*-nya — memungkinkan data dan perilaku suatu entitas (seperti mahasiswa) dikelompokkan secara terstruktur dalam satu kesatuan.
4. Penerapan *encapsulation* melalui visibilitas *private* beserta method *getter* dan *setter* melindungi data dari perubahan yang tidak sah, sekaligus memungkinkan validasi dilakukan sebelum data disimpan.
5. Namespace berperan penting dalam mencegah konflik penamaan class, khususnya pada aplikasi berskala besar yang melibatkan banyak class dari berbagai sumber.
6. Method tambahan seperti *getAngkatan()* membuktikan bahwa class dapat diperluas dengan logika bisnis sederhana yang memanfaatkan data internalnya sendiri, dalam hal ini menentukan tahun angkatan berdasarkan pola penomoran NIM.
7. Kombinasi antara layout, partial, dan OOP menjadi fondasi penting sebelum mengimplementasikan Controller dan integrasi database pada tahap pengembangan aplikasi selanjutnya.